

PHP/Laravel Interview Prep - Testing & Maintenance Cheat Sheet

1. Testing Strategies

- Manual Testing: For key user flows and UI interactions.
- Laravel Feature/Unit Testing: Use 'php artisan test' to validate logic and edge cases.
- Validation: Input validation on both frontend (JavaScript/HTML) and backend (Laravel Request validation).
- Error Handling: Laravel's built-in logging, try-catch blocks, and error pages.

2. Maintenance Practices

- Version Control: Use Git with branching, commits, and pull requests.
- Refactoring: Clean and improve code structure regularly for better readability and performance.
- Dependency Updates: Keep composer.json and composer.lock up-to-date (e.g., 'composer update').
- Backups: Automate database and file backups.
- Documentation: Maintain README files, code comments, and setup instructions.

Flashcards

Q: What testing methods do you use in Laravel applications?

A: Manual testing for user flows, Laravel's feature and unit tests for business logic validation.

Q: How do you ensure input validation in Laravel?

A: By using form request classes or the validate() method in controllers.

Q: How do you handle error tracking and debugging?

A: Using Laravel logs (storage/logs), custom error pages, and debugging tools like Laravel Debugbar.

Q: What are your best practices for maintaining applications?

A: Version control with Git, refactoring code, regular backups, updating dependencies, and documentation.

Q: How do you update PHP dependencies in Laravel?

A: Using 'composer update' or 'composer require package-name'.